

SQL Subqueries — Reference Guide

All 7 types, with real business applications, using the orders / users e-commerce tables

1 Scalar Subquery

Returns exactly one value. Used directly with =, >, <, or inside an expression.

Application: Find orders priced above the store's average — useful for flagging high-value orders for priority handling.

```
SELECT order_id, total_price
FROM orders
WHERE total_price > (SELECT AVG(total_price) FROM orders);
```

2 Single-row Subquery

Same as scalar — returns one row. Fails with an error if the subquery unexpectedly returns more than one row.

Application: Customer support looks up a specific customer's order history by their unique email.

```
SELECT * FROM orders
WHERE user_id = (SELECT user_id FROM users WHERE email = 'john@example.com');
```

Using a non-unique column like name here will throw "Subquery returns more than 1 row" if duplicates exist.

3 Multi-row Subquery

Returns a list of values. Must be used with IN, ANY, or ALL — never =.

Application: A regional manager wants every order placed by customers linked to a physical outlet.

```
SELECT * FROM orders
WHERE user_id IN (SELECT user_id FROM users WHERE outlet_id IS NOT NULL);
```

4 Multi-column Subquery

Returns more than one column, compared together as a matched pair.

Application: Match orders where both the customer *and* status align with an existing high-value order — avoids false positives from matching only one column.

```
SELECT * FROM orders o1
WHERE (o1.user_id, o1.status) IN (
    SELECT user_id, status FROM orders WHERE total_price > 1000
);
```

5 Correlated Subquery

The inner query depends on the outer query's current row, so it re-runs once per row and can't execute standalone.

Application: Marketing wants to email only customers who have actually placed an order — skipping signups who never bought anything.

```
SELECT name FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

6 Nested Subquery

A subquery inside another subquery — each layer solves a smaller piece feeding the one above it.

Application: Identify the single biggest-spending customer by name for a "customer of the month" feature.

```
SELECT name FROM users
WHERE user_id = (
    SELECT user_id FROM orders
    WHERE total_price = (SELECT MAX(total_price) FROM orders)
);
```

7 Derived Table (Subquery in FROM)

A subquery placed in the FROM clause, treated as a temporary table for the outer query to filter on.

Application: Find the average order value per region, but only show regions averaging over \$500 — filtering on a calculated value that GROUP BY alone can't filter directly in WHERE.

```
SELECT region_summary.address, region_summary.avg_price
FROM (
    SELECT users.address, AVG(orders.total_price) AS avg_price
    FROM orders
    JOIN users ON orders.user_id = users.user_id
    GROUP BY users.address
) AS region_summary
WHERE region_summary.avg_price > 500;
```

Type	Returns	Used with
Scalar	1 row, 1 column	=, >, <
Single-row	1 row, 1 column	=, >, <
Multi-row	Many rows, 1 column	IN, ANY, ALL
Multi-column	1+ rows, multiple columns	IN with column pairs
Correlated	Depends on outer row	EXISTS, or any operator
Nested	Varies per layer	Any — layered
Derived table	Full result set (table-like)	FROM clause

SQL Subqueries Reference — built from orders / users e-commerce schema